MSL-UNet: High-Performance U-Net Based on Vision Mamba and Strip Large-Kernel Convolution

Zhibo Wang and Kaigui Wu
School of Computer Science, Chongqing University, Chongqing, 400044, China

March 3, 2026

Abstract

1 Introduction

Automated segmentation of anatomical structures in medical images, particularly computed tomography (CT), is a foundational task for computer-aided diagnosis, treatment planning and quantitative analysis [1, 2]. Convolutional Neural Networks (CNNs) long enjoyed a dominant position due to their exceptional feature extraction capabilities. The U-Net architecture, based on CNNs, further established a new design paradigm and became the cornerstone for numerous models. Over time, however, the inherent limitation of CNNs in modeling long-range dependencies increasingly hindered their application in medical practice. Researchers addressed this issue by introducing the self-attention mechanism, and built up high-performing models such as TransUNet [3] and SwinUNet [4]. Nonetheless, the substantial computational overhead associated with self-attention imposes a significant burden on today's healthcare industry, creating a pressing demand for a new technological solution. This demand is satisfied by Vision Mamba, a novel architecture built upon selective state space models (SSMs) [5, 6].

Vision Mamba achieves competitive visual modeling capability with linear computational complexity, offering an efficient paradigm for capturing global context. Initial explorations have successfully integrated Mamba blocks as encoders in segmentation frameworks [7], demonstrating their potential. Nevertheless, a critical gap persists: The decoding layer depends on leveraging excellent contextual capabilities for generation, which is evidently not the inherent forte of Mamba. This explains why prevalent vision models built upon Mamba predominantly utilize it as the encoder, integrating it with a separate, non-Mamba decoder, as seen in architectures like Samba and MaIL.

To bridge this gap, we propose MSL-UNet, a novel hybrid architecture for efficient and accurate CT organ segmentation. Our core innovation lies in the first dedicated integration of a Vision Mamba-based encoder with a comprehensively enhanced convolutional decoder. Specifically, we construct the encoder using hierarchical Vision Mamba (VSS) blocks to efficiently build global feature representations. For the decoder, we introduce a new Multi-scale Strip Large-kernel Convolutional Block (MSLCB) and a Strip Large-Kernel Grouped Attention Gate (SLGAG) to replace the standard components.The MSLCB employs parallel strip convolutions and large-kernel depth-wise separable convolutions to capture anisotropic features and broad contextual information simultaneously. The SLGAG refines the fusion of encoder and decoder features through skip connections using the same enhanced convolutional philosophy, ensuring that crucial spatial details and directional cues are preserved during upsampling.

The principal contributions of this work are threefold:

1. We introduce MSL-UNet that synergistically combines the global modeling strength of a Vision Mamba encoder with the localized, detail-oriented power of a heavily augmented convolutional decoder.

2. We design two novel core components MSLCB and SLGAG that systematically incorporate strip and large-kernel convolutions to better address the geometric and textural characteristics of medical organs.

3. Extensive experiments on Synapse and ACDC demonstrate that our MSL-UNet achieves state-of-the-art performance with markedly improved efficiency, offering a compelling solution for practical medical image analysis.

2 Related Works

In this section we give a brief review of related works in the realm of medical image segmentation before introducing Mamba and then give a short review of SSMs.

2.1 Medical image segmentation

The evolution of medical image segmentation has been fundamentally driven by one core challenge: the trade-off between efficient local feature extraction and effective global context modeling. Early vision architectures predominantly relied on Convolutional Neural Networks (CNNs), with the U-Net framework establishing the de facto standard. However, even if CNNs are highly efficient and inductive bias-friendly, the intrinsic locality of convolution operations still limits its ability to capture long-range dependencies, which decided that CNNs naturally do poor performance in response to more complex scences.

To address this limitation of long-range dependency modeling, researchers introduced Vision Transformer (ViTs), which brought a paradigm shift to the field of computer vision. By replacing convolutions with self-attention mechanism, Transformer demonstrated remarkable success in capturing global contextual information across the entire image, which granted models capacity to feel an entire image. The advent of Transformer has spurred the development of numerous image segmentation models built upon the self-attention mechanism. TransFuse uses a parallel encoder to allow both CNN and ViT to process local and global features in the same time. Swin-UNet further enhanced efficiency and adaptability to dense prediction tasks through hierarchical feature maps and shifted window attention. DS-TransUNet can capture multi-scale feature from patches of varying sizes with a two parallel Swin Transformers decoder.

However, this leap in modeling capacity came at a significant cost: the self-attention mechanism scales quadratically with input sequence length, leading to substantial computational and memory burdens, especially for high-resolution 2D or 3D medical images. That is why researchers start building hybrid architectures,which can slightly release performance burden. One seminal example is TransUNet that employs a CNN network as a feature map extractor in its encoder and then utilizes a Transformer to model global dependencies at the bottleneck. TransUNet also equips the model with capabilities for both feature extraction and global perception, yet it achieves higher performance than pure Transformer models. This clearly demonstrates the greater potential of hybrid architectures, which is precisely why many current vision models adopt such a design. Our model is also based on hybrid architecture.

2.2 State Space Model

State Space Models (SSMs), rooted in classical control theory and signal processing, provide a powerful framework for modeling dynamical systems. They describe the evolution of a system's hidden state over time through linear differential (or difference) equations, offering a principled approach for processing continuous or discrete sequences. Traditionally, SSMs were valued for their strong theoretical foundations and ability to capture long-range dependencies with linear computational complexity, a stark contrast to the quadratic scaling of self-attention in Transformers.

The application of SSMs to deep learning gained significant momentum with the introduction of the Structured State Space Sequence model (S4). S4 combined HiPPO theory for long-range memory with a structured, linear time-invariant (LTI) formulation, enabling efficient training and demonstrating remarkable performance on long-sequence modeling tasks. However, a key limitation of standard S4 and similar LTI-SSMs is their time-invariant nature—their system parameters are fixed and independent of the input, which restricts their ability to perform context-dependent reasoning or selectively focus on relevant information within a sequence.

This fundamental limitation was addressed by the groundbreaking selective state space model, exemplified by Mamba. The core innovation of Mamba lies in making the SSM parameters functions of the input, thereby introducing a selection mechanism. This allows the model to dynamically decide what information to propagate or ignore, significantly enhancing its expressiveness for discrete, information-dense data like language and images. This selective mechanism, while maintaining the computational benefits of linear scaling, is the key that unlocked the high performance of Mamba-based architectures in domains previously dominated by Transformers.

3 Methodology

In this section we firstly introduce the overall architecture of our MSL-UNet and then explain the used modules.

3.1 MSL-UNet

As depicted in Figure 1 (a), the overall architecture of MSL-UNet is presented, which is based on U-Shape-Network. Specifically, MSL-UNet comprises a Patch Embedding layer as initial progressing, a four-stage VSS Blocks as encoder, a four-stage MSLCBs as decoder, a SH layer as final projection and SLGAGs as skip connections.

Firtsly, in the Patch Embedding layer input image is devided from $x = R^{H \times W \times 3}$ into $x' \in R^{\frac{H}{4} \times \frac{W}{4} \times C}$, each patch is non-overlapping and the size is 4×4, subsequently mapping the dimensions

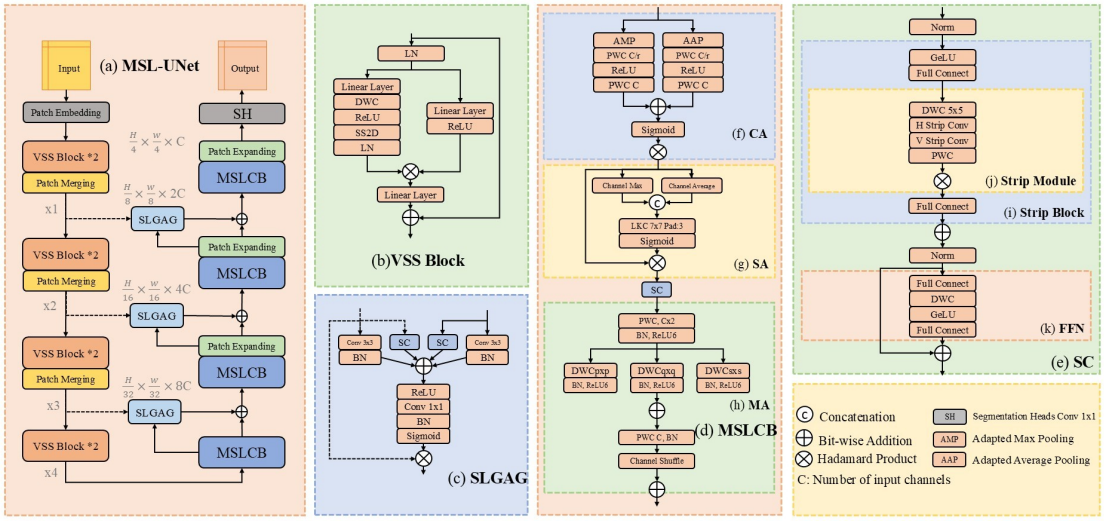


Figure 1: (a) The overall architecture of MSL-UNet. (b) VSS Block is the main construction of our encoder. (c) Strip Large-kernel grouped attention gate(SLGAG) can enhance features from skip connections. (d) Multi-scale Strip Large-kernel convolutional block(MSLCB), progressing patches in all dimensions to further enhance features, is main construction of our decoder. (e) Strip convolutional block(SC). (f) Channel attention block(CA). (g) Spatial attention block(SA). (h) Multi-scale attention block(MA). (i) Strip block. (j) Strip module. (k) Feed-Forward Network(FFN).

of the image to C. Before entering the encoder layers, x' will be normalized.

For the encoder, we employ eight VSS Blocks in four stages where one stage pocesses two VSS Blocks, with a patch merging operation at the end of the first three statges to reduce the height and width of the input features while increasing the number of channels. The channel counts for each stage is [C, 2C, 4C, 8C].

Similarly, for the decoder we employ four MSLCBs to refine pyramid features extracted from the encoder via skip connections(i.e., x1,x2,x3,x4 in Figure1). Also, a patch expanding operation is used at the end of the last three MSLCBs. The channel counts for each stage is [8C, 4C, 2C, C].

We employ three SLGAGs for the skip connections. Features that get upsampled in patch expanding layer adds output from SLGAGs.

Finally, the SH layer produces the final segmentaion output.

3.2 VSS Block

As depicted in Figure 1 (b), the VSS Block, deriving from VMamba [8], is the core module of our encoder. Firstly, the input patches x will get a layer normalized, then split into two branches, where one branch does global modeling but losses local feature and the other branch preseves local feature.

The front branch firstly process the input patches through a linear layer $LL(.)$ and a depth-wise separable convolution $DWC(.)$, with a activation function $R(.)$ at the end. We then feed the patches into the $SS2D(.)$ for furture feature extraction. Finally, the features get a layer normalized $LN(.)$ and await output patches from the other branch.

As the front branch losses local features through a global modeling, we preseve local features in the other branch where patches will only undergoes a linear layer $LL(.)$ and a activation function $ReLU(.)$.

Both of the output from two branches then get hadamard producted after one more linear layer $LL(.)$. Finally, the merged features get a bit-wise addition with initial input x.

The $VSSBlock(.)$ can be formulated as in Equations 1 to 4:

$$Bch_1(x) = LN(SS2D(R(DWC(LL(LN(x))))))) \quad (1)$$

$$Bch_2(x) = R(LL(LN(x))) \quad (2)$$

$$Mrg(x) = LL(Bch_1(x) \otimes Bch_2(x)) \quad (3)$$

$$VSSBlock(x) = x + Mrg(x) \quad (4)$$

3.3 Strip Large-kernel grouped attention gate (SLGAG)

We introduce a new strip large-kernel group attention gate(SLGAG) to get higher activation of relevant features and suppression of irrelevant ones. As shown in Figure 1 (c), SLGAG accepts two inputs g and x separately from a patch expanding layer in the decoder and a patch merging layer that lies one level higher than the former one, both of the input have same size to guarantee bit-wise addition opeates normally. We process g and x with 3×3

convolution $Cnv_{3\times3}(.)$ and strip convolution $SC(.)$ to get feature from different dimensions. These convolved features then get batch normalized $BN(.)$ and merged through bit-wise addition. The merged feature map is activated in ReLU layer $R(.)$. Afterward, we apply a 1×1 convolution block $Cnv_{1\times1}(.)$ to get a singal-channel feature map. Then we exsuccessively pass the singal-channel feature map through a $BN(.)$ and a activation function $\sigma(.)$ to yield the attention coefficients. Finally, we apply hadamard product to x and the activated singal-channel feature map, producting attention-gated feature.

The $SLGAG(.)$ can be formulated as in Equations 5 to 8:

$$add(.) = SC(.) + BN(Cnv_{3\times3}(.)) \quad (5)$$

$$Add(g, x) = add(g) + add(x) \quad (6)$$

$$Smd(g, x) = \sigma(BN(Cnv_{1\times1}(R(Add(g, x))))) \quad (7)$$

$$SLGAG(g, x) = x \otimes Smd(g, x) \quad (8)$$

3.4 Multi-scale Strip Large-kernel convolutional block (MSLCB)

We introduce a new multi-scale strip large-kernel convolutional block as depicted in Figure 1 (d). Our MSLCB consits of a channel attention block (CA) that finds most relevant channel, a spatial attention block (SA) that find the most relevant parts feature map, a strip convolution block (SC) that enhances feature extraction capacity for tiny details, and a multi-scale attention block (MA) that enhances contextual relationships among all of the features.

Channel Attention Block (CA): As depicted in Figure 1 (f), the input patches x firtsly enter CA then split into two branches where x gets separately adapted max poooling $AMP(.)$ and adapted average pooling $AAP(.)$ to extract the most significant feature of the entire feature map per channel. For each pooled feature maps, we apply two point-wise convolution $PWC_{C/r}(.)$ (i.e., r = 16) and $PWC_C(.)$ separately reduce and recover the number of channels, with a activation function $R(.)$ between them. Afterward, both of the pooled feature maps get a bit-wise addition before a activation function $\sigma(.)$. Finally, we apply a hadamard producting to the pooled feature maps and initial input x.

The $CA(.)$ can be formulated as in Equations 9 to 12:

$$z = z(x) \quad (9)$$

$$Bch(z) = PWC_C(R(PWC_{C/r}(z))) \quad (10)$$

$$Add(x) = Bch(AMP(x)) + Bch(AAP(x)) \quad (11)$$

$$CA(x) = x \otimes \sigma(Add(x)) \quad (12)$$

Spatial Attention Block (SA): As depicted in Figure 1 (g), similarly designed as CA, the input patches x firstly get max pooling $CH_{max}(.)$ and average pooling $CH_{average}(.)$ in two different branches along the channel dimension to pay attention to local features, then get contacted before entering a large-kernel convolution layer $LKC(.)$ (i.e., size is 7×7 and padding is 3), where local contextual relationships among the features will get enhanced. Finally, the contacted feature maps will undergo into an activation function $\sigma(.)$ and be hadamard producted with initial input x.

The $SA(.)$ can be formulated as in Equations 13 and 14:

$$Con(x) = [CH_{max}(x), CH_{average}(x)] \quad (13)$$

$$SA(x) = x \otimes \sigma(LCK(Con(x))) \quad (14)$$

Multi-scale Attention Block (MA): As depicted in Figure 1 (h), the input patches x firstly get a channel number expanding (i.e., expanding factor is 2) in a point-wise convolution $PWC_{C\times2}(.)$, followed by a batch normalization $BN(.)$ and a activation function $R6(.)$. Then the patches are separately passed through three depth-wise separable convolution $DWC(.)$ layers in order to capture both multi-scale and multi-resolution contexts. After feature from different channel getting summed, we use a channel shuffle $CSF(.)$ layer to incorporate relationships among channels. The shuffled feature maps then undergo into another $PWC_C(.)$ where number of channels get recovered, with a batch normalization layer $BN(.)$ after that. Finally, normalized feature maps get a bit-wise addition with initial input x.

The $MA(.)$ can be formulated as in Equations 15:

$$Con(x) = [CH_{max}(x), CH_{average}(x)] \quad (15)$$

$$SA(x) = x \otimes \sigma(LCK(Con(x))) \quad (16)$$

3.5 Strip Convolution Block (SC)

Strip Convolution Block (SC) is designed as shown in Figure 1 (e). Given input x will firstly pass through a Norm Layer, a Full connect Layer and a activation function GeLU, then where the normalized patches $x^{'}$ next enter is Strip Module, which is the kernel part of SC depicted in Figure 1 (j).

In Strip Module, a depth-wise convolution with large kernel(i.e., 5×5) is first applied to extract local contextual features. Then two sequential depth-wise convolution of strip-shaped kernels are used to better capture objects of high aspect rations, with a point-wise convolution followed.

The output of Strip Module, denoted as Z, will pass through a hadamard producting with $x^{'}$, yielding $Z^{'}$, which then pass through a bit-wise addition with x after a full connection layer. Afterward, full connected output will be applied a bit-wise addition again with x, yielding Y. In the next normalized Y

Methods	DICE↑	HD95↓	Aor.	Gal.	Kid.(L)	Kid.(R)	Liv.	Pan.	Spl.	Sto.
UNet	70.11	44.69	84.00	56.70	72.41	62.64	86.98	48.73	81.48	67.96
AttnUNet	71.70	34.47	82.61	61.94	76.07	70.42	87.54	46.70	80.67	67.66
SSFormer	78.01	25.72	82.78	63.74	80.72	78.11	93.53	61.53	87.07	76.61
PolypPVT	78.08	25.61	82.34	66.14	81.21	73.78	94.37	59.34	88.05	79.4
Swin-UNet	79.13	21.55	85.47	66.53	83.28	79.61	94.29	56.58	90.66	76.60
TransDeepLab	80.16	21.25	86.04	69.16	84.08	79.88	93.53	61.19	89.00	78.40
MT-UNet	78.59	26.59	87.92	64.99	81.47	77.29	93.06	59.46	87.75	76.81
MEW-UNet	78.92	16.44	86.68	65.32	82.87	80.02	93.63	58.36	90.19	74.26
PVT-CASCADE	81.06	20.23	83.01	70.59	82.23	80.37	94.08	64.43	90.1	83.69
VM-UNet	81.08	19.21	86.40	69.41	86.16	82.76	94.17	58.80	89.51	81.40
MSL-UNet(ours)	82.56	16.18	87.8	67.21	88.07	84.55	95.16	62.59	93.25	81.81

Table 1: Results on Synapse Multi-Organ Segmentation. DICE scores are reported for individual organs. ↑denotes higher the better, ↓denotes lower the better. The best results are in bold.

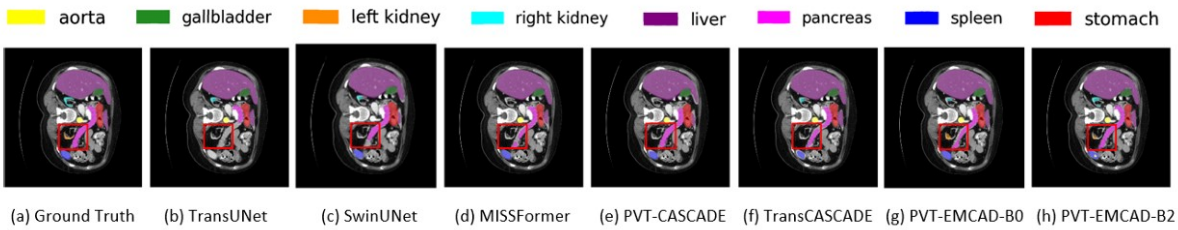


Figure 2: Qualitative results on Synapse datasets are shown .The rectangular box highlights incorrectly segmented polyps by SOTA methods.

Methods	DICE	RV	Myo	LV
TransUNet	89.71	86.67	87.27	95.18
SwinUNet	90.00	88.55	85.62	95.83
HiFormer	90.12	91.06	84.54	94.77
TransDeepLab	90.41	91.45	84.54	95.23
MT-UNet	90.43	86.64	89.04	95.62
MISSFormer	90.86	89.55	88.04	94.99
ParaTransCNN	91.31	92.76	85.84	95.34
TransCASCADE	91.63	90.25	89.14	95.50
VM-UNet	92.12	90.65	89.68	96.02
MSL-UNet(ours)	92	92	92	92

Table 2: Results on ACDC. DICE scores(%)are reported for individual organs. The best results are in bold.

undergoes into a feed-forward network where features get refined and enhanced, with a bit-wise addition with normalized Y after that, yielding final output $SC(X)$.

3.6 Loss Function

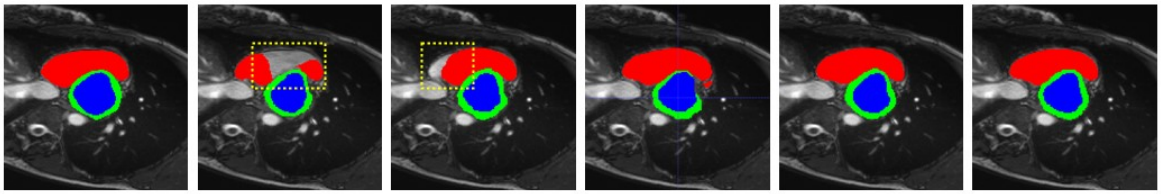
4 Experiments

4.1 Implementation details

We use both Syanpse and ACDC to train and test, with all of the images are resized to 224×224. We employ data augmentation techniques, including randomflipand random rotation to prevent overfitting. AdamW optimizer with an initial learning rate and weight decay of 1e-3 is utilized for training. CosineAnnealingLR is utilized as the scheduler with a maximum of 50 iterations and a minimum learning rate of 1e-5. We set the training epochs to 300. Two primary metrics: the DICE Score and the 95% Hausdorff Distance (HD95) are utilized to evaluate our model. All experiments are conducted using a single NVIDIA RTX 5090 GPU.

4.2 Experimental Results

We compare our MSL-UNet with many state-of-the-art models on Syanpse and ACDC datasets. We choose models with different type of architecture. Pure CNN models represented by classic UNet. Pure Transformer models like Swin-UNet. Pure Mamba models like VM-UNet. Hybrid architecture models like TransUNet that uses CNN and Transformer. And other SOTA models such as PVT-CASCADE and so on. Notbly, we specially compare our MSL-UNet with VM-UNet, which similarly uses VSS Block as encoder, however, also as decoder. We believe Mamba is better used as encoder but not fitting as decoder, later results will show this.



(a) Ground Truth (b) Swin-Unet (c) TransUnet (d) TransDeepLab (e) VM-UNet (f) MSL-Unet(ours)

Figure 3: Qualitative results on ACDC datasets are shown .Red represents the right ventricle, blue represents the left ventricle,and green represents the myocardium. The yellow rectangular box highlights incorrectly segmented polyps by SOTA methods.

4.2.1 Results on Synapse Multi-Organ Segmentation

Results of experiments on Synapse are presented in Table 1. In SOTAs using different type of architecture, our MSL-UNet achieves the best performance, boasting a mean DICE score of 82.56 and a mean HD95 of 16.18, surpassing PVT-CASCADE and VM-UNet. We believe two facts help our model, one is Mamba encoder costs less resources, the other one is our MSL-UNet employs large-kernel and strip convolution that successfully enhance capacity of feature extraction. Figure ℂred1 shows visual results of some SOTA models on Synapse segmentation. It's clear to see our MSL-UNet performs better in segmentation on tiny and linear organs, such as right kidney. Some SOTA models incorrectly predict, especially in right kindney, and other organs.

4.2.2 Results of ACDC Segmentation

Results of experiments on ACDC are presented in Table 2. Our model achieves the best performance with a mean DICE score of 92.37. Figure ℂred2 presents visual results of SOTA models performing on ACDC segmentation. It's clear that our MSL-UNet get a better performance than Swin-UNet, TransUNet and TransDeepLab. Swin-UNet predict incorrectly, missing the middle dark part of left ventricle. TransUNet similarly miss the left part. TransDeepLab miss parts of Myo and captures wrong extra parts of left ventricle. Additionally, VM-UNet predicts around correctly, however, not smoothly or precisely as our MSL-UNet.

4.3 Ablation Studies

4.3.1 Effect of Mamba Encoder

We conduct a set of experiments on the Synapse datasets to understand eth effect of different encoders. We have applied pure CNN, pure Transformer as encoder to replace mamba encoder of our MSL-UNet. Table NO. shows the experimental results.

Methods	DICE	RV
CNN-SL-Unet	99	99
Transformer-SL-UNet	99	99
MSL-UNet(Ours)	99	99

Table 3: Results on Syanpse of models using different encoders.

5 Conclusions

In this paper, we have presented MSL-UNet, a new

References

[1] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention* (pp. 234–241).

[2] Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2), 203–211.

[3] Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., ... & Zhou, Y. (2021). TransUNet: Transformers make strong encoders for medical image segmentation. *arXiv preprint arXiv:2102.04306*.

[4] Cao, H., Wang, Y., Chen, J., Jiang, D., Zhang, X., Tian, Q., & Wang, M. (2021). Swin-Unet: Unet-like pure transformer for medical image segmentation. *arXiv preprint arXiv:2105.05537*.

[5] Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

[6] Liu, Y., Tian, Y., Zhao, Y., Yu, H., Xie, L., Wang, Y., ... & Ye, Q. (2024). VMamba: Visual State Space Model. *arXiv preprint arXiv:2401.10166*.

[7] Ruan, J., & Xiang, S. (2024). VM-UNet: Vision Mamba UNet for medical image segmentation. *arXiv preprint arXiv:2402.02491*.

[8] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, and Y. Liu, "Vmamba: Visual state space model," arXiv preprint arXiv:2401.10166, 2024.